A_dynamical_model_fo...
PDF

explain this

The paper **"A Dynamical Model for Generating Synthetic Electrocardiogram Signals"** by McSharry, Clifford, Tarassenko, and Smith (IEEE Transactions on Biomedical Engineering, 2003) introduces a **mathematical model** that simulates realistic ECG (electrocardiogram) waveforms. Here's a structured explanation:

🩺 1. Purpose of the Model

The authors created a **dynamical system** capable of producing **synthetic ECG signals** that resemble real human ECGs.
This is useful for:

- Testing and validating biomedical signal processing algorithms (e.g., R-peak detection, QT-interval analysis).
- Controlling signal characteristics (mean heart rate, variability, morphology, noise).
- Creating benchmark datasets for clinical and research purposes.

⚙️ 2. Model Overview

The ECG is generated by **three coupled ordinary differential equations (ODEs)** that describe the trajectory of a point moving in **three-dimensional space (x, y, z)**:

- The trajectory circles around a **limit cycle** in the (x, y) plane (each revolution = one heartbeat).
- Vertical motion (z) produces the **PQRST waveform** — the characteristic ECG shape.

Each component of the ECG (P, Q, R, S, T) corresponds to specific attractors/repellors in this 3D system:

- When the trajectory nears one of these points, it is "pushed" or "pulled" in z, producing the appropriate deflection (wave) in the ECG.

💓 3. Heart Rate Variability (HRV)

The model incorporates realistic **beat-to-beat variability** using heart rate modulation derived from physiological processes:

- **High-frequency (HF)** oscillations (~0.25 Hz): due to **respiratory sinus arrhythmia (RSA)**.
- **Low-frequency (LF)** oscillations (~0.1 Hz): due to **Mayer waves** (blood pressure regulation).

The ratio **LF/HF** reflects **sympathovagal balance**, i.e., the balance between sympathetic and parasympathetic nervous activity.

🧮 4. Equations and Simulation

- The system's angular velocity controls how fast the trajectory moves around the circle — this sets the **RR interval** (time between heartbeats).
- The RR intervals are generated with a **bimodal Gaussian power spectrum** to mimic natural HRV (Equation 3 in the paper).
- Integration of the ODEs is done numerically (Runge–Kutta method).

Noise (e.g., Gaussian random noise) can be added to simulate real-world measurement errors.

📊 5. Results

The model:

- Produces realistic ECG signals with natural-looking **PQRST complexes**.
- Shows modulation effects (e.g., changes in R-peak amplitude) due to HRV.
- Demonstrates physiological relationships:
  - **QT-interval increases linearly** with **RR-interval** (as in real ECGs).
  - **RS-amplitude correlates** with **RR-intervals**.
- Enables evaluation of **ECG-derived respiratory signals (EDR)** and **noise robustness** in signal processing.

🔬 6. Applications and Extensions

The model can be extended for:

1. **Personalized ECGs** — fit parameters to a person's morphology and HRV.
2. **Testing noise filtering** and **artefact removal** algorithms.
3. **Simulating cardiac abnormalities** (e.g., ST elevation, QT prolongation, ectopic beats).
4. **Generating multilead ECGs** — by mapping (x, y, z) to different electrode configurations.

🧠 7. Key Takeaway

This is a **biophysically inspired, data-driven model** that captures:

- The **geometry** of ECG waveform generation.
- The **dynamics** of heart rate variability.
- The **statistical control** of heart rate and morphology.

It bridges physiological realism and mathematical control, providing a benchmark tool for **testing and validating biomedical signal analysis algorithms**.

Would you like me to show the **actual equations** (the differential system and the RR modulation formula) and explain them step-by-step with visuals or code (e.g., Python simulation)?

explain the mathematics of that

Let's unpack the **mathematics** behind the synthetic ECG model in the paper. The model is elegant because it uses **a simple set of three coupled ordinary differential equations (ODEs)** to capture both the **morphology** (shape) of each ECG beat and the **temporal variability** (beat-to-beat timing).

🧭 1. Overview: ECG as a Trajectory in 3D Space

The model imagines the ECG signal as the **z-coordinate** of a moving point (x, y, z) in three-dimensional state space:

- The point moves **around a circle** (limit cycle) in the x–y plane.
- Each revolution around the circle corresponds to **one heartbeat**.
- The **vertical motion** $z(t)$ produces the P, Q, R, S, and T deflections.

This approach gives both **quasi-periodicity** (because the trajectory loops repeatedly) and **variability** (because the angular speed varies with heart-rate changes).

⚙️ 2. The Core Dynamical System

The dynamics are defined by a system of ODEs (equation (1) in the paper):

$$\begin{cases} \dot{x} = \alpha x - \omega y, \\ \dot{y} = \alpha y + \omega x, \\ \dot{z} = -\sum_i a_i \Delta\theta_i e^{-\frac{\Delta\theta_i^2}{2b_i^2}} - (z - z_0), \end{cases}$$

where:

- $r = \sqrt{x^2 + y^2}$,
- $\alpha = 1 - r$ (restores the trajectory to the unit circle → the limit cycle),
- $\omega(t)$ is the **angular velocity**, controlling how fast we go around the circle,
- z_0 is the **baseline** (which can oscillate with respiration).

Thus:

- The first two equations define rotation on a circle with radius 1.
- The third equation defines vertical "kicks" in z at specific angles corresponding to the ECG features.

📍 3. Generating the P, Q, R, S, T Waves

Each event (P, Q, R, S, T) is represented by a Gaussian bump centered at an angular position θ_i:

$$\Delta\theta_i = \mathrm{mod}(\theta - \theta_i + \pi,\, 2\pi) - \pi,$$

so that $\Delta\theta_i$ measures the angular distance between the current position and the event.

Each event has parameters a_i and b_i:

- a_i: amplitude (positive = upward, negative = downward),
- b_i: width (controls sharpness of the deflection).

Typical parameter values are chosen empirically to match a real ECG morphology (given in Table I of the paper).

As the trajectory passes each angular position θ_i, the z-equation adds a Gaussian-shaped deviation, generating the PQRST waveform (see Fig. 3 in the paper).

⏱️ 4. Heart-Rate Dynamics: Varying the Angular Velocity

The **angular velocity** $\omega(t)$ determines how quickly the trajectory moves around the circle, i.e., the **RR interval**.

To reproduce natural variability, the RR intervals are generated from a **power spectrum** with two Gaussian peaks:

$$S(f) = e^{-\frac{(f-f_{\mathrm{LF}})^2}{2\sigma_{\mathrm{LF}}^2}} + e^{-\frac{(f-f_{\mathrm{HF}})^2}{2\sigma_{\mathrm{HF}}^2}},$$

where:

- $f_{\mathrm{LF}} \approx 0.1\,\mathrm{Hz}$ (Mayer waves),
- $f_{\mathrm{HF}} \approx 0.25\,\mathrm{Hz}$ (respiratory sinus arrhythmia).

Then, a random phase Fourier synthesis produces a time series of RR intervals:

$$RR(t) = \mathrm{IFFT}\left(\sqrt{S(f)}\, e^{i\phi(f)}\right),$$

where $\phi(f)$ are random phases uniformly distributed in $[0, 2\pi]$.

Finally, the instantaneous angular velocity is:

$$\omega(t) = \frac{2\pi}{RR(t)}.$$

This makes each revolution correspond to a different heartbeat length, with the correct physiological frequency modulation.

5. Baseline Wander

To include **baseline drift** due to breathing, a sinusoidal component is added:

$$z_0(t) = A \sin(2\pi f_{\text{resp}} t),$$

where A is small (e.g. 0.15 mV) and f_{resp} is the respiratory frequency.
This gives slow undulations like those seen in real ECGs.

---

6. Numerical Integration

The equations are integrated using a **fourth-order Runge–Kutta method** with time step $1/f_s$, where f_s is the sampling frequency (e.g. 256 Hz or 360 Hz).

The output signal $z(t)$ after integration is the synthetic ECG.

Noise can be added as Gaussian measurement noise:

$$z_{\text{noisy}}(t) = z(t) + \mathcal{N}(0, \sigma^2),$$

typically with $\sigma = 0.025\,\text{mV}$.

---

7. Emergent Properties

Mathematically, this compact system reproduces several observed ECG features:

- **QT–RR correlation** arises naturally because slower heart rates (longer RR) give more time between Q and T.
- **R-peak amplitude modulation** correlates with RR variability due to the geometric motion in z.
- The **LF/HF ratio** of HRV can be tuned directly by adjusting the power spectrum $S(f)$.

---

8. Summary of Mathematical Structure

Component	Mathematical Representation	Physiological Meaning
Limit cycle	$(\dot{x}, \dot{y}) = (\alpha x - \omega y,\ \alpha y + \omega x)$	Periodic heart rhythm
ECG morphology	$\dot{z} = -\sum a_i \Delta\theta_i e^{-\Delta\theta_i^2/2b_i^2} - (z - z_0)$	PQRST waveform
HRV control	$\omega(t) = 2\pi/RR(t)$, $S(f)$ with LF/HF bands	Sympathovagal modulation
Respiration	$z_0(t) = A\sin(2\pi f_{\text{resp}} t)$	Baseline wander
Noise	$z_{\text{noisy}} = z + \mathcal{N}(0, \sigma^2)$	Measurement error

---

Would you like me to **derive these equations step-by-step from the physical assumptions** (starting from the idea of a limit cycle oscillator) or show **how to implement them numerically (e.g., in Python)** to generate a synthetic ECG?

is this code okay :

```
function [x, dxdt] = ecg_synth(t)
% ECG_SYNTH   Generate synthetic ECG signals.
%   X = ECG_SYNTH(T), where T is a vector of time values, returns a vector
%   containing the samples of a synthetic ECG signal (in mV),
%   corresponding to the given time values. The signal is generated using
%   the mathematical model described in:
%       P. E. McSharry, G. D. Clifford, L. Tarassenko and L. A. Smith,
%       "A dynamical model for generating synthetic electrocardiogram
%       signals," IEEE Transactions on Biomedical Engineering, vol. 50,
%       no. 3, pp. 289–294, 2003.
```

```
%   without baseline wander and assuming heart rate = 1
Hz.
%
%   [X, DXDT] = ECG_SYNTH(T) also returns the vector
of values of the first
%   derivative of the generated synthetic ECG signal.
%
%   Jakub Wagner, Warsaw University of Technology,
August 2025

    if ~isvector(t) || numel(t) < 3
        error('Input argument must be a vector with at least
3 elements.')
    end
    HR = 1.0;  % heart rate [Hz]

    % Parameters of P, Q, R, S and T waves
    theta = [-0.33, -0.08, 0, 0.08, 0.5] * pi; % position
within cycle [rad]
    a = [1.2, -5.0, 30.0, -7.5, 0.75];        % related to height
    b = [0.25, 0.1, 0.1, 0.1, 0.4];           % related to width

    % Generate signal by solving ODEs
    omega = 2*pi*HR;
    f = @(~, v) ode(v, omega, theta, a, b);
    v0 = [-1, 0, -0.005]';  % initial conditions selected
empirically
    [~, v] = ode45(f, t, v0);

    x = zeros(size(t));  % x will have the same direction
(row/column) as t
    x(:) = v(:,3);

    % Compute first derivative if needed
    if nargout >= 2
        dxdt = zeros(size(x));
        for n = 1:numel(x)
            dvdt = ode(v(n,:)', omega, theta, a, b);
            dxdt(n) = dvdt(3);
        end
```

```
    end

    % Auxiliary function representing the set of ordinary
differential
    % equations implementing the adopted model of ECG
signals
    function dvdt = ode(v, omega, theta, a, b)
        theta0 = atan2(v(2), v(1));
        dtheta = rem(theta0 - theta, 2*pi);
        dvdt = [
            -omega * v(2);
            omega * v(1);
            -v(3) - sum(a .* dtheta .* exp(-0.5 * (dtheta ./
b).^2))
            ];
    end
end
```

Thought for 1m 12s

Short answer: it will *run* and it will look ECG-ish, but it's not quite the McSharry model as published, and there are a couple of mathematical details I'd tighten up.

I'll go through the main points and then show a slightly "more faithful" version.

---

1. What your code is doing (mathematically)

Inside the nested `ode` you have:

matlab Copy code

```matlab
theta0 = atan2(v(2), v(1));
dtheta = rem(theta0 - theta, 2*pi);
dvdt = [
    -omega * v(2);
    omega * v(1);
    -v(3) - sum(a .* dtheta .* exp(-0.5 * (dtheta ./ b).^2))
    ];
```

So:

- You treat $(x, y) = (v(1), v(2))$ as a point on the unit circle.
- You set

  $$\dot{x} = -\omega y, \quad \dot{y} = \omega x,$$

  which is uniform circular motion (constant radius).
- You compute the angle

  $$\theta = \operatorname{atan2}(y, x),$$

  and the angular distances to each P/Q/R/S/T angle θ_i:

  $$\Delta\theta_i = \operatorname{rem}(\theta - \theta_i, 2\pi).$$
- Then you set

  $$\dot{z} = -z - \sum_i a_i \, \Delta\theta_i \exp\left(-\frac{\Delta\theta_i^2}{2b_i^2}\right),$$

  which is essentially McSharry's

  $$\dot{z} = -\sum_i a_i \, \Delta\theta_i e^{-\Delta\theta_i^2/(2b_i^2)} - (z - z_0),$$

  with $z_0 = 0$ (no baseline wander).

That's all consistent in spirit with the paper and will generate a reasonable synthetic ECG for a constant heart rate.

---

2. Differences vs. the original McSharry model

(a) Missing radial "restoring term"

In the paper the x, y dynamics are:

$$\dot{x} = \alpha x - \omega y, \quad \dot{y} = \alpha y + \omega x, \quad \alpha = 1 - r, \; r = \sqrt{x^2 + y^2}.$$

That α term pulls the trajectory back to the unit circle and makes the limit cycle attracting.

You have:

matlab Copy code

```
dvdt(1) = -omega * v(2);
dvdt(2) =  omega * v(1);
```

This keeps the radius **exactly constant** if you start on the unit circle (which you do with `v0 = [-1, 0, ...]`). That's mathematically fine and stable enough in practice, but it's technically a simplification.

✅ It's okay for a clean, constant-HR toy model.
👌 If you want to match the paper more closely, add the α term:

matlab Copy code

```
r = hypot(v(1), v(2));
alpha = 1 - r;
dvdt = [
    alpha * v(1) - omega * v(2);
    alpha * v(2) + omega * v(1);
    -v(3) - sum(a .* dtheta .* exp(-0.5 * (dtheta ./ b).^2))
];
```

(b) Angle wrapping (`dtheta`)

In the paper, the angular difference is defined so it stays in $(-\pi, \pi]$:

$$\Delta\theta_i = \mathrm{mod}(\theta - \theta_i + \pi,\, 2\pi) - \pi.$$

You currently do:

matlab Copy code

```
dtheta = rem(theta0 - theta, 2*pi);
```

Issues:

- `rem` in MATLAB doesn't wrap to $(-\pi, \pi]$; it can give values in $(-2\pi, 2\pi)$.
- That means for differences near $\pm\pi$ you're not using the shortest angular distance, which the model assumes.

Will it break everything?

Not catastrophically – far from each event, the Gaussian factor kills the contribution anyway – but it's the one genuinely "wrong" mathematical detail if you're aiming for the published model.

Better: use a proper wrapped angle, e.g.:

matlab Copy code

```matlab
dtheta = atan2(sin(theta0 - theta), cos(theta0 - theta));
```

This gives Δθ in $(-\pi, \pi]$ and matches the model definition.

(c) Heart rate / RR variability

You hard-code:

matlab Copy code

```matlab
HR = 1.0;    % heart rate [Hz]
omega = 2*pi*HR;
```

So ω is **constant**, meaning:

- **No heart rate variability**,
- No RSA or Mayer wave effects,
- Just one clean PQRST per second.

That's consistent with your comment *"without baseline wander and assuming heart rate = 1 Hz"* 👍 — just be aware it's a **very** simplified version of McSharry, which normally drives $\omega(t)$ from a synthetic RR series.

(d) Derivative computation

You recompute the derivative in a loop:

matlab Copy code

```
    if nargout >= 2
        dxdt = zeros(size(x));
        for n = 1:numel(x)
            dvdt = ode(v(n,:)', omega, theta, a, b);
            dxdt(n) = dvdt(3);
        end
    end
```

Mathematically this is fine: `dxdt(n)` is exactly $\dot{z}$ evaluated on the numerical trajectory at that time.

It's just a bit inefficient; if performance ever matters you could:

- Vectorize it, or
- Use `deval` on the `ode45` solution, or
- Ask the user whether they really need `dxdt` .

But correctness-wise: ✅ all good.

---

3. Minor style/robustness notes

Not "wrong", but worth thinking about:

- Make sure `t` is **sorted and increasing**; `ode45` expects that:

  matlab Copy code

  ```
  if any(diff(t) <= 0)
      error('t must be strictly increasing.');
  end
  ```

- Maybe rename the nested function from `ode` to something like `ecg_rhs` to avoid confusion with MATLAB's `ode*` functions.
- Starting point `v0 = [-1, 0, -0.005]` is fine; it just sets the initial phase on the circle and a tiny initial z offset.

---

4. A "tweaked" version keeping your structure

Here's your function with the two main mathematical tweaks (α term + angle wrapping) but otherwise unchanged:

matlab Copy code

```matlab
function [x, dxdt] = ecg_synth(t)
    if ~isvector(t) || numel(t) < 3
        error('Input argument must be a vector with at least 3 elements.')
    end

    HR = 1.0;  % heart rate [Hz]

    % Parameters of P, Q, R, S and T waves
    theta_i = [-0.33, -0.08, 0, 0.08, 0.5] * pi; % [rad]
    a       = [1.2, -5.0, 30.0, -7.5, 0.75];      % heights
    b       = [0.25, 0.1, 0.1, 0.1, 0.4];         % widths

    omega = 2*pi*HR;
    rhs = @(~, v) ecg_rhs(v, omega, theta_i, a, b);
    v0 = [-1, 0, -0.005]';  % initial conditions (on the unit circle)

    [~, v] = ode45(rhs, t, v0);
    x = zeros(size(t));
    x(:) = v(:,3);  % ECG is z-component

    if nargout >= 2
        dxdt = zeros(size(x));
        for n = 1:numel(x)
            dvdt = ecg_rhs(v(n,:)', omega, theta_i, a, b);
            dxdt(n) = dvdt(3);
        end
    end

    function dvdt = ecg_rhs(v, omega, theta_i, a, b)
        x = v(1); y = v(2); z = v(3);

        % Limit-cycle dynamics with radial restoring term
        r = hypot(x, y);
        alpha = 1 - r;

        theta0 = atan2(y, x);
```

```
        % Wrapped angular distance in (-pi, pi]
        dtheta = atan2(sin(theta0 - theta_i), cos(theta0 - theta_i));

        dzdt = -z - sum(a .* dtheta .* exp(-0.5 * (dtheta ./ b).^2));

        dvdt = [
            alpha * x - omega * y;
            alpha * y + omega * x;
            dzdt
        ];
    end
end
```

---

5. Bottom line

- ✅ Your code is **essentially correct** for a **simplified** constant-HR, no-baseline version and will generate a reasonable synthetic ECG.
- 🧪 If you want it to be mathematically faithful to McSharry et al.:
  - Add the radial term $\alpha = 1 - r$,
  - Use a proper wrapped angle for `dtheta`,
  - (Optionally) later drive `omega(t)` from an RR-series if you want HRV.

If you'd like, I can help you extend this to include **heart-rate variability** (LF/HF structure) or **baseline wander** next.

Your code is essentially correct for a simplified constant-HR, no-baseline version and will generate a reasonable synthetic ECG.

🧪 If you want it to be mathematically faithful to McSharry et al.:

Add the radial term
α
=

1
–
r
α=1–r,

Use a proper wrapped angle for dtheta,

(Optionally) later drive omega(t) from an RR-series if you want HRV.

add also this

Thought for 29s ›

Nice, let's wire that in properly.

You already had a good constant-HR implementation. Below is an updated version that:

1. **Adds the radial term** $\alpha = 1 - r$ → attracting limit cycle (McSharry-style).
2. **Uses a properly wrapped angle** for `dtheta` (shortest angular distance, in $(-\pi, \pi]$).
3. **Drives** $\omega(t)$ **from a time-varying heart rate** with LF and HF components (simple HRV: a "Mayer wave" at 0.1 Hz + RSA-like term at 0.25 Hz).

Still no baseline wander, as you requested.

matlab Copy code

```
function [x, dxdt] = ecg_synth(t)
% ECG_SYNTH   Generate synthetic ECG signals with simple HRV.
%   X = ECG_SYNTH(T), where T is a vector of time values, returns a vector
%   containing samples of a synthetic ECG signal (in mV), corresponding to
%   the given time values. The signal is generated using a dynamical model
%   based on:
%       P. E. McSharry, G. D. Clifford, L. Tarassenko and L. A. Smith,
%       "A dynamical model for generating synthetic electrocardiogram
%       signals," IEEE Trans. Biomed. Eng., vol. 50, no. 3, pp. 289–294, 2
%
%   This version:
```

```
%     * uses an attracting limit cycle (alpha = 1 - r),
%     * uses a wrapped angular distance dtheta in (-pi, pi],
%     * includes simple HRV by modulating heart rate with LF and HF terms.
%
%   [X, DXDT] = ECG_SYNTH(T) also returns the first derivative of the
%   generated synthetic ECG signal.
%
%   Jakub Wagner, Warsaw University of Technology, August 2025
%   (modified with McSharry-style dynamics + simple HRV)

    % Basic input check
    if ~isvector(t) || numel(t) < 3
        error('Input argument must be a vector with at least 3 elements.')
    end
    t = t(:);   % ensure column; direction is restored at the end

    if any(diff(t) <= 0)
        error('Time vector t must be strictly increasing.');
    end

    % --- Heart rate parameters (in Hz) ---
    HR_mean = 1.0;   % mean heart rate: 1 Hz = 60 bpm

    % Simple HRV components:
    f_LF = 0.10;     % low-frequency (~Mayer waves)
    f_HF = 0.25;     % high-frequency (~respiration / RSA)
    A_LF = 0.10;     % LF modulation amplitude [Hz]
    A_HF = 0.05;     % HF modulation amplitude [Hz]

    % Instantaneous HR as a function of time
    HR_fun = @(tau) HR_mean + ...
                    A_LF * sin(2*pi*f_LF*tau) + ...
                    A_HF * sin(2*pi*f_HF*tau);

    % --- P, Q, R, S, T parameters (McSharry-style) ---
    theta_i = [-0.33, -0.08, 0, 0.08, 0.5] * pi; % angular positions [rad]
    a       = [ 1.2,  -5.0, 30.0, -7.5, 0.75];    % "heights"
    b       = [ 0.25,  0.10, 0.10, 0.10, 0.40];   % "widths"

    % Initial conditions (on unit circle, small z offset)
```

```
v0 = [-1, 0, -0.005]';   % [x0; y0; z0]

% ODE right-hand side with time-varying omega(t), radial term, wrapped
rhs = @(tau, v) ecg_rhs(tau, v, HR_fun, theta_i, a, b);

% Solve ODE system
[~, v] = ode45(rhs, t, v0);

% ECG signal is z-component
x = v(:, 3);

% Restore direction (row/column) as original t
if isrow(t)
    x = x.';
end

% First derivative if requested
if nargout >= 2
    dxdt = zeros(size(x));
    for n = 1:numel(x)
        dvdt = ecg_rhs(t(n), v(n,:).', HR_fun, theta_i, a, b);
        dxdt(n) = dvdt(3);
    end
    if isrow(t)
        dxdt = dxdt.';
    end
end

% ------------------------------------------------------------------
% Nested function: right-hand side of ODEs
% ------------------------------------------------------------------
function dvdt = ecg_rhs(tau, v, HR_fun, theta_i, a, b)
    x_ = v(1);
    y_ = v(2);
    z_ = v(3);

    % Time-varying angular frequency from HR(tau)
    HR = HR_fun(tau);
    if HR < 0.2
        HR = 0.2;  % safety clamp
```

```
        end
        omega = 2*pi*HR;

        % Radial coordinate and restoring term: alpha = 1 - r
        r = hypot(x_, y_);
        alpha = 1 - r;

        % Current angular position on the circle
        theta0 = atan2(y_, x_);

        % Shortest angular distance to each event in (-pi, pi]
        dtheta = atan2(sin(theta0 - theta_i), cos(theta0 - theta_i));

        % z-dynamics (no baseline wander: z0 = 0)
        dzdt = -z_ - sum(a .* dtheta .* exp(-0.5 * (dtheta ./ b).^2));

        dvdt = [
            alpha * x_ - omega * y_;   % dx/dt
            alpha * y_ + omega * x_;   % dy/dt
            dzdt                       % dz/dt
        ];
    end
end
```

What changed vs. your original

- **Radial term** added:

  matlab Copy code

  ```
  r = hypot(x_, y_);
  alpha = 1 - r;
  dvdt(1) = alpha*x_ - omega*y_;
  dvdt(2) = alpha*y_ + omega*x_;
  ```

  → pulls the trajectory onto the unit circle (true limit cycle).

- **Wrapped angle** for `dtheta` :

  matlab Copy code

  ```
  dtheta = atan2(sin(theta0 - theta_i), cos(theta0 - theta_i));
  ```

→ gives Δθ in (–π, π], matching the model's "shortest angular distance" assumption.

- **Time-varying ω(t)** from a simple HRV model:

```matlab
HR_fun = @(tau) HR_mean + A_LF*sin(2*pi*f_LF*tau) + A_HF*sin(2*pi*f_HF
HR = HR_fun(tau);
omega = 2*pi*HR;
```

→ LF and HF modulation of heart rate instead of constant HR.

If you want, next step I can help you:

- tune `A_LF`, `A_HF` to match a target HRV std, or
- replace the sinusoidal HRV with a **true RR-process from a shaped power spectrum** (closer to McSharry's exact spectral construction).

so I have this code:

clear; clc; close all;

%% Parameters from McSharry & Clifford (TBME 2003) / ECGSYN

sfecg = 256; % ECG sampling frequency [Hz]
N = 256; % approximate number of heart beats
Anoise = 0; % additive measurement noise amplitude [mV]
hrmean = 60; % mean heart rate [bpm]
hrstd = 1; % std of heart rate [bpm]
lfhfratio = 0.5; % LF/HF ratio
sfint = 512; % internal sampling frequency [Hz]

% PQRST morphology parameters (order: P Q R S T)
% These are exactly the values given in the paper / ECGSYN code.
ti = [-70 -15 0 15 100]; % angles of extrema

```
[degrees]
ai = [ 1.2 -5 30 -7.5 0.75];    % z-position (amplitudes) of
extrema
bi = [0.25 0.1 0.1 0.1 0.4];    % Gaussian width of peaks

%% Call ECGSYN (McSharry–Clifford model)
% NOTE: ecgsyn() expects ti in *degrees* and converts
internally to radians.
[s, ipeaks] = ecgsyn(sfecg, N, Anoise, hrmean, hrstd,
lfhfratio, sfint, ti, ai, bi);

%% Time vector for ECG
t = (0:length(s)-1)/sfecg;

%% 1) Time-domain ECG
figure;
plot(t, s, 'LineWidth', 1.2);
xlabel('Time (s)');
ylabel('ECG (mV)');
title('Synthetic ECG from ECGSYN (McSharry–Clifford)');
grid on;

% Optional: mark R peaks
R = find(ipeaks == 3);
hold on;
plot(t(R), s(R), 'o', 'MarkerSize', 4);

%% 2) Phase-plane: z vs dz/dt
% Approximate derivative using finite differences
dt   = 1/sfecg;
dsdt = gradient(s, dt);

figure;
plot(s, dsdt, 'LineWidth', 1.0);
xlabel('ECG amplitude s (mV)');
ylabel('ds/dt (mV/s)');
title('Phase-plane portrait: s vs ds/dt');
grid on;
```

now lets do this :

ii. Parameter Variation Analysis: Explore how changing parameters ai and bi (amplitude and width) impacts the ECG morphology. Experiment with different values and discuss the observed effects.

Thought for 15s >

Awesome, this is a perfect place to poke at the model and see how it "draws" the ECG. Let's do two things:

1. Explain **what ai and bi do mathematically**.
2. Give you **ready-to-run MATLAB code** to vary them and visually compare morphologies, plus some guidance on how to interpret what you see.

---

1. What do ai and bi actually control?

In the McSharry model, the z-dynamics (the ECG amplitude) is driven by Gaussian "bumps" at angular positions corresponding to P, Q, R, S, T:

$$\dot{z} = -z - \sum_{i \in \{P,Q,R,S,T\}} a_i \, \Delta\theta_i \exp\left[-\frac{\Delta\theta_i^2}{2b_i^2} \right]$$

- a_i: amplitude term → controls **height/depth** of that wave
  - Larger |a_R| → taller R peak.
  - More negative a_Q → deeper Q, etc.
- b_i: width term → controls **how broad or sharp** that wave is in phase
  - Smaller b_R → very sharp, narrow QRS complex.
  - Larger b_T → wider, more "rounded" T wave.

So:

- **ai = [aP aQ aR aS aT]**
- **bi = [bP bQ bR bS bT]**

They map very directly to what you *see* on the ECG.

---

2. Experiment 1 – Vary amplitudes ai

Let's keep everything you wrote, and then add a block that varies ai and plots the effect on one or a few beats.

We'll:

- Change **one wave at a time** (e.g. R, T, P).
- Compare original vs modified morphologies.

matlab Copy code

```matlab
%% Baseline parameters (your existing setup)
clear; clc; close all;

sfecg     = 256;   % ECG sampling frequency [Hz]
N         = 256;   % approximate number of heart beats
Anoise    = 0;     % additive measurement noise amplitude [mV]
hrmean    = 60;    % mean heart rate [bpm]
hrstd     = 1;     % std of heart rate [bpm]
lfhfratio = 0.5;   % LF/HF ratio
sfint     = 512;   % internal sampling frequency [Hz]

% PQRST morphology parameters (P Q R S T)
ti = [-70 -15 0 15 100];        % [deg]
ai = [ 1.2 -5 30 -7.5 0.75];    % amplitudes
bi = [0.25 0.1 0.1 0.1 0.4];    % widths

% Generate baseline signal
[s_base, ipeaks] = ecgsyn(sfecg, N, Anoise, hrmean, hrstd, lfhfratio, sfin
t = (0:length(s_base)-1)/sfecg;

%% Take a short window with a few beats for clearer morphology
twin = t(t <= 5);   % first 5 seconds
swin_base = s_base(1:length(twin));

figure;
plot(twin, swin_base, 'k', 'LineWidth', 1.2);
xlabel('Time (s)'); ylabel('ECG (mV)');
title('Baseline ECG (first 5 s)');
grid on;

%% ---- Parameter Variation: Amplitudes ai ----
```

```
% Example: vary R-wave amplitude (3rd element of ai)
scale_factors = [0.5 1.0 1.5];  % reduce, baseline, increase

figure;
colors = lines(numel(scale_factors));
for k = 1:numel(scale_factors)
    ai_mod = ai;
    ai_mod(3) = ai(3) * scale_factors(k);   % modify R only

    [s_mod, ~] = ecgsyn(sfecg, N, Anoise, hrmean, hrstd, lfhfratio, sfint,
    swin_mod = s_mod(1:length(twin));

    plot(twin, swin_mod, 'Color', colors(k,:), 'LineWidth', 1.2); hold on;
end
xlabel('Time (s)');
ylabel('ECG (mV)');
legend('R x0.5','R x1.0','R x1.5');
title('Effect of changing R-wave amplitude a_R');
grid on;

%% Example: vary T-wave amplitude (5th element of ai)
scale_factors = [0.2 1.0 2.0];

figure;
colors = lines(numel(scale_factors));
for k = 1:numel(scale_factors)
    ai_mod = ai;
    ai_mod(5) = ai(5) * scale_factors(k);   % modify T only

    [s_mod, ~] = ecgsyn(sfecg, N, Anoise, hrmean, hrstd, lfhfratio, sfint,
    swin_mod = s_mod(1:length(twin));

    plot(twin, swin_mod, 'Color', colors(k,:), 'LineWidth', 1.2); hold on;
end
xlabel('Time (s)');
ylabel('ECG (mV)');
legend('T x0.2','T x1.0','T x2.0');
title('Effect of changing T-wave amplitude a_T');
grid on;
```

What you should see

- Increasing **a_R**:
  - Taller R peak, deeper RS segment (because S follows R).
  - QRS complex dominates even more visually.
- Decreasing **a_R**:
  - Smaller R peak; ECG looks “low voltage” in the QRS.
- Increasing **a_T**:
  - Larger T wave, more obvious repolarization.
  - If you make it too big, T can look pathologically tall.
- Decreasing **a_T**:
  - Flatter T wave; could resemble low-repolarization amplitude.

You can repeat similar experiments for **a_P, a_Q, a_S**:

- Increase **a_P** → more prominent atrial depolarization (P).
- Make **a_Q** more negative → deeper initial Q notch.
- Make **a_S** more negative → deeper S, more “spiky” terminal part of QRS.

---

3. Experiment 2 – Vary widths bi

Now let’s see what happens when we change the **temporal sharpness** of waves.

We’ll vary:

- **b_R** → QRS duration / sharpness
- **b_T** → T-wave breadth

matlab Copy code

```matlab
%% ---- Parameter Variation: Widths bi ----
% Example: vary R-wave width (3rd element of bi)
width_factors = [0.5 1.0 2.0];  % narrower, baseline, broader

figure;
colors = lines(numel(width_factors));
for k = 1:numel(width_factors)
    bi_mod = bi;
    bi_mod(3) = bi(3) * width_factors(k);   % modify R width only
```

```
    [s_mod, ~] = ecgsyn(sfecg, N, Anoise, hrmean, hrstd, lfhfratio, sfint,
    swin_mod = s_mod(1:length(twin));

    plot(twin, swin_mod, 'Color', colors(k,:), 'LineWidth', 1.2); hold on;
end
xlabel('Time (s)');
ylabel('ECG (mV)');
legend('b_R x0.5','b_R x1.0','b_R x2.0');
title('Effect of changing R-wave width b_R');
grid on;

%% Example: vary T-wave width (5th element of bi)
width_factors = [0.5 1.0 2.0];

figure;
colors = lines(numel(width_factors));
for k = 1:numel(width_factors)
    bi_mod = bi;
    bi_mod(5) = bi(5) * width_factors(k);   % modify T width only

    [s_mod, ~] = ecgsyn(sfecg, N, Anoise, hrmean, hrstd, lfhfratio, sfint,
    swin_mod = s_mod(1:length(twin));

    plot(twin, swin_mod, 'Color', colors(k,:), 'LineWidth', 1.2); hold on;
end
xlabel('Time (s)');
ylabel('ECG (mV)');
legend('b_T x0.5','b_T x1.0','b_T x2.0');
title('Effect of changing T-wave width b_T');
grid on;
```

What you should see

- **b_R smaller (e.g., ×0.5)**:
  - Narrow, very sharp QRS.
  - More high-frequency content (steep up/down).
- **b_R larger (e.g., ×2)**:
  - Broader QRS complex → mimics bundle branch blocks / conduction delay visually.
- **b_T smaller**:

    - T wave becomes sharper / narrower.
- **b_T larger**:
    - Broad, “domed” T wave; takes more of the ST–T segment.

Again, you can try changing **b_P, b_Q, b_S** similarly:

- Smaller **b_P** → very tight, sharp P.
- Larger **b_S** → more spread-out terminal part of QRS.

---

4. How to *discuss* the observed effects (for a report)

When you write this up, you can structure it like:

> We systematically varied the amplitude parameters a_i and width parameters b_i of the McSharry–Clifford ECG model while keeping all other parameters fixed. The impact of these changes on the ECG morphology was assessed visually in the time domain.

Then describe:

- **Amplitude variation (ai)**
    - “Increasing a_R from 0.5× to 1.5× scaled the height of the R peak while leaving other waves largely unchanged. This resembles changes in QRS voltage (e.g., due to chest impedance or ventricular mass).”
    - “Increasing a_T produced higher T waves, mimicking increased ventricular repolarization amplitude.”
- **Width variation (bi)**
    - “Reducing b_R produced a narrow, sharp QRS, while increasing b_R widened the complex, analogous to conduction delays (e.g., bundle branch block).”
    - “Changing b_T altered the duration of the T wave, affecting the apparent QT morphology.”

You can also mention that:

- Changing ai and bi **does not significantly affect heart rate variability**, since HRV is controlled separately via the RR-process; it mainly reshapes each beat.
- Extreme values of ai or bi can cause **overlapping waves** (e.g., T encroaching on next P), which is useful to test algorithm robustness but less physiological.

If you tell me which exact variations you *must* cover (e.g., “only T and QRS” or “all five waves”), I can tighten the code to match that spec and help you phrase the discussion more like an assignment answer.

give the codes in matlab

Here’s a clean MATLAB script you can drop in and run (assuming `ecgsyn.m` is on your path). It:

1. Generates a **baseline** ECG.
2. Varies **ai** (P, Q, R, S, T amplitudes) one-by-one.
3. Varies **bi** (P, Q, R, S, T widths) one-by-one.

Everything is sectioned so you can run parts separately in the Editor.

matlab Copy code

```matlab
%% ECG Parameter Variation: ai (amplitudes) and bi (widths)
% Requires ecgsyn.m (McSharry–Clifford model) on the MATLAB path.

clear; clc; close all;

%% ------------------------------------------------------------------
%  Base parameters (McSharry & Clifford / ECGSYN)
% ------------------------------------------------------------------
sfecg     = 256;   % ECG sampling frequency [Hz]
N         = 256;   % approximate number of heart beats
Anoise    = 0;     % additive measurement noise amplitude [mV]
hrmean    = 60;    % mean heart rate [bpm]
hrstd     = 1;     % std of heart rate [bpm]
lfhfratio = 0.5;   % LF/HF ratio
sfint     = 512;   % internal sampling frequency [Hz]

% PQRST morphology parameters (order: P Q R S T)
ti = [-70 -15 0 15 100];        % angles of extrema [degrees]
ai = [ 1.2 -5 30 -7.5 0.75];    % amplitudes
bi = [0.25 0.1 0.1 0.1 0.4];    % widths
```

```
% Generate baseline synthetic ECG
[s_base, ipeaks] = ecgsyn(sfecg, N, Anoise, hrmean, hrstd, ...
                          lfhfratio, sfint, ti, ai, bi);

t  = (0:length(s_base)-1)/sfecg;
twin = t(t <= 5);                       % first 5 seconds
swin_base = s_base(1:length(twin));   % windowed baseline

% Plot baseline
figure;
plot(twin, swin_base, 'k', 'LineWidth', 1.2);
xlabel('Time (s)'); ylabel('ECG (mV)');
title('Baseline synthetic ECG (first 5 s)');
grid on;

%% ------------------------------------------------------------------------
%  1) Amplitude variation: ai
%     We scale each ai element (P, Q, R, S, T) separately.
% ------------------------------------------------------------------------
scale_factors = [0.5 1.0 1.5];    % 50%, 100%, 150%
wave_names    = {'P','Q','R','S','T'};

for w = 1:5     % loop over P, Q, R, S, T
    figure;
    colors = lines(numel(scale_factors));

    for k = 1:numel(scale_factors)
        ai_mod = ai;
        ai_mod(w) = ai(w) * scale_factors(k);   % modify only one wave

        [s_mod, ~] = ecgsyn(sfecg, N, Anoise, hrmean, hrstd, ...
                            lfhfratio, sfint, ti, ai_mod, bi);
        swin_mod = s_mod(1:length(twin));

        plot(twin, swin_mod, 'Color', colors(k,:), 'LineWidth', 1.2); hold
    end

    plot(twin, swin_base, 'k--', 'LineWidth', 1.0);  % baseline for refere
    xlabel('Time (s)'); ylabel('ECG (mV)');
```

```
    legend('0.5x','1.0x','1.5x','baseline','Location','best');
    title(sprintf('Effect of changing amplitude a_%s', wave_names{w}));
    grid on;
end

%% ------------------------------------------------------------------------
%  2) Width variation: bi
%     We scale each bi element (P, Q, R, S, T) separately.
% ------------------------------------------------------------------------
width_factors = [0.5 1.0 2.0];   % narrower, baseline, broader

for w = 1:5     % loop over P, Q, R, S, T
    figure;
    colors = lines(numel(width_factors));

    for k = 1:numel(width_factors)
        bi_mod = bi;
        bi_mod(w) = bi(w) * width_factors(k);   % modify only one width

        [s_mod, ~] = ecgsyn(sfecg, N, Anoise, hrmean, hrstd, ...
                            lfhfratio, sfint, ti, ai, bi_mod);
        swin_mod = s_mod(1:length(twin));

        plot(twin, swin_mod, 'Color', colors(k,:), 'LineWidth', 1.2); hold
    end

    plot(twin, swin_base, 'k--', 'LineWidth', 1.0);  % baseline reference
    xlabel('Time (s)'); ylabel('ECG (mV)');
    legend('0.5x','1.0x','2.0x','baseline','Location','best');
    title(sprintf('Effect of changing width b_%s', wave_names{w}));
    grid on;
end

%% ------------------------------------------------------------------------
%  Optional: Phase-plane portrait for one chosen case (e.g. modified R)
% ------------------------------------------------------------------------
% Example: R amplitude 1.5x
ai_R15 = ai; ai_R15(3) = ai(3)*1.5;
[s_R15, ~] = ecgsyn(sfecg, N, Anoise, hrmean, hrstd, ...
                    lfhfratio, sfint, ti, ai_R15, bi);
```

```
s_R15_win = s_R15(1:length(twin));
dt = 1/sfecg;
dsdt_R15 = gradient(s_R15_win, dt);

figure;
plot(s_R15_win, dsdt_R15, 'LineWidth', 1.0);
xlabel('ECG amplitude s (mV)');
ylabel('ds/dt (mV/s)');
title('Phase-plane: s vs ds/dt (R amplitude 1.5x)');
grid on;
```

You can comment out sections you don't need, or tweak `scale_factors` / `width_factors` if you want more extreme changes.

iii. Incorporating Heart Rate Variability (HRV): Introduce heart rate variability by setting ω as a function of time. Use a sinusoidal modulation for ω to simulate changing heart rates. Plot the modified ECG and discuss the effects of HRV on the signal.

Thought for 13s

Got it! Here's a self-contained MATLAB script that adds **heart-rate variability (HRV)** by making the angular speed $\omega(t)$ a **sinusoid of time**, then plots (i) ECG without HRV vs with HRV, and (ii) the derived RR/instantaneous-HR so you can see the modulation effects.

It uses the McSharry-style ODE (limit-cycle with P/Q/R/S/T Gaussians), adds the radial restoring term $\alpha = 1 - r$, and uses a properly wrapped angular distance. You don't need `ecgsyn.m` for this—just paste and run.

matlab Copy code

```
%% HRV via sinusoidal omega(t) in a McSharry-style ECG ODE
clear; clc; close all;
```

```
%% Sampling & simulation horizon
sf   = 256;           % output sampling [Hz]
T    = 15;            % seconds
t    = linspace(0, T, T*sf).';  % column vector

%% Morphology parameters (P Q R S T)
% Angles (radians) correspond roughly to [-70 -15 0 15 100] degrees
ti = (pi/180)*[-70 -15 0 15 100];
ai = [ 1.2  -5.0  30.0  -7.5  0.75];
bi = [ 0.25  0.10  0.10  0.10  0.40];

%% Two heart-rate profiles:
% 1) Baseline (no HRV): HR(t) = HR_mean
% 2) HRV: HR(t) = HR_mean + HR_amp * sin(2*pi*f_HRV*t)
HR_mean = 60/60;      % 60 bpm -> 1 Hz
HR_amp  = 12/60;      % ±12 bpm -> 0.2 Hz amplitude around mean (tweakable
f_HRV   = 0.10;       % 0.10 Hz (Mayer-wave-like) modulation (tweakable)

HR_const = @(tau) HR_mean;
HR_sin   = @(tau) max(0.3, HR_mean + HR_amp * sin(2*pi*f_HRV*tau)); % clam

%% Integrate ODE for both cases
x0 = [-1; 0; -0.005];          % initial state on unit circle, small z
s0 = integrate_ecg_ode(t, x0, ti, ai, bi, HR_const);  % no HRV
sH = integrate_ecg_ode(t, x0, ti, ai, bi, HR_sin);    % with HRV

%% Plot time-domain ECG (first 10 s)
twin = t <= 10;
figure;
plot(t(twin), s0(twin), 'k', 'LineWidth', 1.1); hold on;
plot(t(twin), sH(twin), 'r', 'LineWidth', 1.1);
xlabel('Time (s)'); ylabel('ECG (mV)');
title('ECG without HRV (black) vs with HRV (red)');
legend('No HRV','With HRV','Location','best'); grid on;

%% R-peak detection and instantaneous HR
minRRsec = 0.3;             % guard for peak distance (~200 bpm upper boun
minPkDist = round(minRRsec * sf);
```

```
[R0pks, R0loc] = findpeaks(s0, 'MinPeakDistance', minPkDist);
[RHpks, RHloc] = findpeaks(sH, 'MinPeakDistance', minPkDist);

tR0 = t(R0loc);
tRH = t(RHloc);

RR0 = diff(tR0);            % seconds
RRH = diff(tRH);            % seconds
HR0 = 60 ./ RR0;            % bpm
HRH = 60 ./ RRH;            % bpm

%% Plot RR and instantaneous HR (with HRV)
figure;
subplot(2,1,1);
stem(tR0(2:end), RR0, 'k', 'filled'); hold on;
stem(tRH(2:end), RRH, 'r', 'filled');
ylabel('RR (s)'); title('Beat-to-beat RR intervals'); grid on;
legend('No HRV','With HRV','Location','best');

subplot(2,1,2);
plot(tR0(2:end), HR0, 'k-o', 'LineWidth', 1.0, 'MarkerSize', 3); hold on;
plot(tRH(2:end), HRH, 'r-o', 'LineWidth', 1.0, 'MarkerSize', 3);
xlabel('Time (s)'); ylabel('Inst. HR (bpm)');
title('Instantaneous HR derived from R-peaks');
grid on; legend('No HRV','With HRV','Location','best');

%% Optional: overlay detected R-peaks back on the ECG-with-HRV
figure;
plot(t(twin), sH(twin), 'Color',[0.2 0.2 0.2], 'LineWidth', 1.1); hold on;
inWin = tRH <= 10;
plot(tRH(inWin), sH(RHloc(inWin)), 'ro', 'MarkerSize', 4, 'LineWidth', 1);
xlabel('Time (s)'); ylabel('ECG (mV)');
title('ECG with HRV + detected R-peaks'); grid on;

%% ---------------- Helper: ODE integrator ----------------
function s = integrate_ecg_ode(t, x0, ti, ai, bi, HR_fun)
    % McSharry-style ODE with alpha=1-r and wrapped dtheta
    % z(t) (3rd state) is the ECG signal.
    rhs = @(tau, v) ecg_rhs(tau, v, ti, ai, bi, HR_fun);
    % Integrate on the provided time vector
```

```
    % Use ode45 with fixed output time points via 't' argument
    opts = odeset('RelTol',1e-6,'AbsTol',1e-8);
    [~, V] = ode45(rhs, t, x0, opts);
    s = V(:,3);     % ECG is z
end

function dv = ecg_rhs(tau, v, ti, ai, bi, HR_fun)
    x = v(1); y = v(2); z = v(3);
    % Time-varying angular frequency from HR(t)
    HR = HR_fun(tau);                % Hz
    omega = 2*pi*HR;                 % rad/s

    % Limit-cycle attraction: alpha = 1 - r
    r = hypot(x, y);
    alpha = 1 - r;

    % Current angle and shortest angular distances to P/Q/R/S/T
    theta  = atan2(y, x);
    dtheta = atan2(sin(theta - ti), cos(theta - ti)); % in (-pi, pi]

    % z-dynamics (no baseline wander z0: set to 0)
    dz = -z - sum( ai .* dtheta .* exp(-0.5*(dtheta./bi).^2) );

    % State derivatives
    dv = [ alpha*x - omega*y;
           alpha*y + omega*x;
           dz ];
end
```

What to look for / How to discuss

- **Time-domain ECG:** with HRV (red), the spacing between R-peaks varies (some beats closer, some farther). You may also notice mild **amplitude modulation** in R/S/T (an emergent effect of the geometry).
- **RR intervals:** with HRV, RR oscillates at the modulation frequency you set (here 0.10 Hz ≈ 10-s rhythm).
- **Instantaneous HR:** tracks the sinusoid you injected into $\omega(t)$. The no-HRV case is flat at ~60 bpm.

- **Physiology note:** a single sinusoid is a toy HRV—useful for demonstrating the mechanism. Real HRV has both **LF (~0.1 Hz)** and **HF (~0.25 Hz, respiration/RSA)** content and a stochastic character. You can get closer by summing two sinusoids or replacing `HR_fun` with a band-limited random process.

Quick variants (try these tweaks)

- Make HRV "respiratory-like" by adding a 0.25 Hz term:

matlab Copy code

```matlab
HR_sin = @(tau) max(0.3, HR_mean ...
              + (12/60)*sin(2*pi*0.10*tau) ...   % LF
              + (6/60)*sin(2*pi*0.25*tau));      % HF
```

- Increase `HR_amp` to exaggerate effects, or change `f_HRV` to see slower/faster waxing and waning of RR.

just use :

ecgsyn instead of use integrate_ecg_ode, ...

Perfect — if you prefer to use the `ecgsyn` function (the official implementation of the McSharry–Clifford model) to simulate **heart rate variability (HRV)**, you can do it directly by varying the **mean heart rate over time** using a **sinusoidal modulation**.

Here's a **ready-to-run MATLAB script** that uses `ecgsyn` to introduce HRV by modulating the heart rate (and hence angular velocity ω):

✅ MATLAB Script: Simulating HRV with `ecgsyn`

matlab Copy code

```matlab
%% HRV simulation using ECGSYN (McSharry–Clifford model)
clear; clc; close all;
```

```
%% Base ECG parameters (same as the paper)
sfecg     = 256;   % ECG sampling frequency [Hz]
N         = 256;   % approximate number of heart beats
Anoise    = 0;     % additive measurement noise amplitude [mV]
hrmean    = 60;    % mean heart rate [bpm]
hrstd     = 1;     % std of heart rate [bpm]
lfhfratio = 0.5;   % LF/HF ratio
sfint     = 512;   % internal sampling frequency [Hz]

% PQRST morphology parameters (order: P Q R S T)
ti = [-70 -15 0 15 100];        % angles [degrees]
ai = [ 1.2 -5 30 -7.5 0.75];    % amplitudes
bi = [0.25 0.1 0.1 0.1 0.4];    % widths

%% ------------------------------------------------------------------------
% Create baseline ECG (constant HR = 60 bpm)
% ------------------------------------------------------------------------
[s_base, ~] = ecgsyn(sfecg, N, Anoise, hrmean, hrstd, lfhfratio, sfint, ti
t_base = (0:length(s_base)-1)/sfecg;

%% ------------------------------------------------------------------------
% Create HRV ECG by sinusoidal modulation of HR(t)
% ------------------------------------------------------------------------
HR_mean = 60;       % average heart rate [bpm]
HR_amp  = 10;       % amplitude of modulation [bpm]
f_HRV   = 0.1;      % frequency of HR modulation [Hz] ~ 10 s rhythm

t_HRV = (0:N-1) / HR_mean * 60;   % time in seconds roughly per beat
HR_t  = HR_mean + HR_amp * sin(2*pi*f_HRV*t_HRV);  % instantaneous HR(t)

% Preallocate ECG signal
s_HRV = [];

% Generate one segment per varying heart rate
for k = 1:length(HR_t)
    hr_now = HR_t(k);
    [s_tmp, ~] = ecgsyn(sfecg, 1, Anoise, hr_now, hrstd, lfhfratio, sfint,
    s_HRV = [s_HRV; s_tmp];
end
```

```
t_HRV_full = (0:length(s_HRV)-1)/sfecg;

%% ------------------------------------------------------------------------
% Plot ECG signals: baseline vs HRV
% ------------------------------------------------------------------------
figure;
plot(t_base(1:4000), s_base(1:4000), 'k', 'LineWidth', 1.0); hold on;
plot(t_HRV_full(1:4000), s_HRV(1:4000), 'r', 'LineWidth', 1.0);
xlabel('Time (s)'); ylabel('ECG (mV)');
title('Synthetic ECG: Constant HR (black) vs HRV (red)');
legend('No HRV','With HRV','Location','best'); grid on;

%% ------------------------------------------------------------------------
% Instantaneous heart rate visualization
% ------------------------------------------------------------------------
figure;
plot(t_HRV, HR_t, 'b', 'LineWidth', 1.5);
xlabel('Time (s)'); ylabel('Heart Rate (bpm)');
title('Sinusoidal Heart Rate Modulation (HRV)');
grid on;

%% ------------------------------------------------------------------------
% Discussion:
% - The red ECG (with HRV) shows varying RR intervals.
% - HR increases when the RR interval shortens (peaks closer together).
% - HR decreases when RR interval lengthens (beats further apart).
% - The modulation frequency (~0.1 Hz) resembles LF component of HRV.
% ------------------------------------------------------------------------
```

🧠 Explanation

- **Heart-rate modulation:**

  The instantaneous heart rate is set as:

$$HR(t) = HR_{\text{mean}} + HR_{\text{amp}} \sin(2\pi f_{\text{HRV}} t)$$

  This makes the angular frequency:

$$\omega(t) = 2\pi \times HR(t)/60$$

vary smoothly with time.

- **Effect on ECG:**
  The result is **varying RR intervals** — the peaks (R-waves) are closer together during higher heart rate and farther apart during lower heart rate.
  You can see the RR-interval "breathing" in the red trace.
- **Modulation frequency (** `f_HRV = 0.1 Hz` **)** mimics *Mayer waves* (sympathetic oscillations).
  You could add a faster 0.25 Hz term to emulate *respiratory sinus arrhythmia (RSA)*:

matlab Copy code

```
HR_t  = HR_mean + HR_amp * sin(2*pi*0.10*t_HRV) + 5*sin(2*pi*0.25*t_HR
```

📈 How to Discuss the Effect

When writing up the results:

> Introducing HRV through sinusoidal modulation of the instantaneous heart rate causes periodic variation in the RR intervals of the ECG signal. During higher heart rates, successive R-peaks occur closer together (shorter RR), while during lower heart rates, they are farther apart (longer RR). The simulated frequency of HRV (0.1 Hz) produces a slow waxing and waning of heart rate, visually evident as a quasi-periodic expansion and compression of the ECG waveform. This mimics physiological HRV components such as Mayer waves or respiratory sinus arrhythmia.

Would you like me to extend this example to include **both LF (0.1 Hz)** and **HF (0.25 Hz)** HRV components together — like in the full McSharry model?

Warning: Integer operands are required for colon operator when used as index.
Warning: Integer operands are required for colon operator when used as index.
Error using /

Matrix dimensions must agree.

Error in ecgsyn>rrprocess (line 193)
ratio = rrstd/xstd;
^^^^^^^^^^^^^^^^^^^

Error in ecgsyn (line 123)
rr0 =
rrprocess(flo,fhi,flostd,fhistd,lfhfratio,hrmean,hrstd,samp
freqrr,Nrr);
^^^
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

Error in untitled2 (line 202)
[s_tmp, ~] = ecgsyn(sfecg, 1, Anoise, hr_now, hrstd,
lfhfratio, sfint, ti, ai, bi);

^^^
^^

Thought for 36s >

You're hitting that error because you're calling `ecgsyn` with `N = 1`. `ecgsyn` internally builds an RR-process in `rrprocess()`, and it needs **enough beats** to form a spectrum and compute a meaningful standard deviation. With `N` too small, you get dimension/STD issues.

Below is a **drop-in MATLAB script** that introduces **sinusoidal HRV** while still using `ecgsyn`. The trick is to generate the ECG in **short windows**, each with a *constant* HR equal to the sinusoid at that window's center, and **each window uses enough beats** (e.g., ≥ 20) so `ecgsyn` stays happy. We then **trim** each window to a fixed duration and **concatenate** them. (For classroom demos this is perfectly fine; if you want ultra-smooth continuity, you can add a tiny crossfade.)

matlab Copy code

```
%% HRV with ECGSYN by windowed HR modulation (no N=1 calls)
clear; clc; close all;

%% Base parameters (McSharry–Clifford / ECGSYN)
```

```
sfecg     = 256;      % ECG sampling frequency [Hz]
Anoise    = 0;        % additive measurement noise amplitude [mV]
hrstd     = 1;        % std of heart rate [bpm] (kept small; HRV from modu
lfhfratio = 0.5;      % LF/HF ratio (used by ecgsyn internally)
sfint     = 512;      % internal sampling frequency [Hz]

% PQRST morphology parameters (order: P Q R S T)
ti = [-70 -15 0 15 100];        % [deg]
ai = [ 1.2 -5 30 -7.5 0.75];    % amplitudes
bi = [0.25 0.1 0.1 0.1 0.4];    % widths

%% HRV settings (sinusoidal HR mean over time)
HR_mean = 60;     % bpm (center)
HR_amp  = 10;     % bpm (peak amplitude of modulation)
f_HRV   = 0.10;   % Hz (e.g., 0.10 ~ Mayer-wave-like)

T_total = 20;     % total desired signal length [s]
T_win   = 2.0;    % window duration [s] to hold HR constant in each call
                  % (short windows follow the sinusoid closely)
nWins   = ceil(T_total / T_win);

% Pre-allocate output ECG
s_all = [];

% Optional: tiny overlap for nicer joins (crossfade not strictly necessary
overlap_sec = 0.0;                      % set to, e.g., 0.1 for a gentle
win_keep_sec = T_win - overlap_sec;     % how much from each window to ke
keep_samples = max(1, round(win_keep_sec * sfecg));  % integer samples to

% For plotting HR(t) reference
t_win_centers = zeros(nWins,1);
HR_win        = zeros(nWins,1);

for w = 1:nWins
    % Center time of this window in seconds
    t_center = (w - 0.5) * T_win;
    t_win_centers(w) = t_center;

    % Sinusoidal HR at this window
    hr_now = HR_mean + HR_amp * sin(2*pi*f_HRV * t_center);
```

```
    hr_now = max(30, hr_now);  % clip to reasonable range (avoid very low
    HR_win(w) = hr_now;

    % How many beats should ecgsyn simulate in this window call?
    % Ensure enough beats for rrprocess (>=20) AND enough time to cover th
    beats_for_time = ceil( (hr_now/60) * (T_win + 2.0) ); % +margin for tr
    N_beats = max(20, beats_for_time);    % keep >=20 beats for stability

    % Generate the ECG for this window
    [s_tmp, ~] = ecgsyn(sfecg, N_beats, Anoise, hr_now, hrstd, lfhfratio,

    % Keep only the first 'keep_samples' samples to assemble continuous ou
    if length(s_tmp) < keep_samples
        % If this ever happens, extend beats (shouldn't with N_beats>=20)
        s_tmp = [s_tmp; zeros(keep_samples - numel(s_tmp),1)];
    end
    s_keep = s_tmp(1:keep_samples);

    % Concatenate
    s_all = [s_all; s_keep];
end

% If we overshot desired length, trim
desired_len = round(T_total * sfecg);
if length(s_all) > desired_len
    s_all = s_all(1:desired_len);
end

% Time vector
t_all = (0:length(s_all)-1) / sfecg;

%% (Optional) Baseline (no HRV) for comparison
N_baseline = ceil( (HR_mean/60) * (T_total + 5) );  % enough beats to cove
[s_base, ~] = ecgsyn(sfecg, N_baseline, Anoise, HR_mean, hrstd, lfhfratio,
s_base = s_base(1:min(end, desired_len));
t_base = (0:length(s_base)-1) / sfecg;

%% Plot ECG: with HRV vs without HRV
figure;
plot(t_base, s_base, 'k', 'LineWidth', 1.0); hold on;
```

```
plot(t_all,  s_all,  'r', 'LineWidth', 1.0);
xlabel('Time (s)'); ylabel('ECG (mV)');
title('ECG: Constant HR (black) vs Sinusoidally Modulated HR (red)');
legend('No HRV','With HRV','Location','best'); grid on;
xlim([0, min(T_total, t_all(end))]);

%% Derive R-peaks and instantaneous HR (from the HRV signal)
minRRsec = 0.3;                         % safeguard for MinPeakDistance
minPkDist = round(minRRsec * sfecg);
[Rpks, Rloc] = findpeaks(s_all, 'MinPeakDistance', minPkDist);
tR = t_all(Rloc);
RR = diff(tR);                          % seconds
HR_inst = 60 ./ RR;                     % bpm
tHR = tR(2:end);

figure;
subplot(2,1,1);
stem(tR(2:end), RR, 'r', 'filled');
ylabel('RR (s)'); grid on;
title('Beat-to-beat RR intervals (with HRV)');

subplot(2,1,2);
plot(tHR, HR_inst, 'r-o', 'LineWidth', 1.0, 'MarkerSize', 3); grid on;
xlabel('Time (s)'); ylabel('Inst. HR (bpm)');
title('Instantaneous HR derived from R-peaks');

%% Show target window HR(t) versus derived HR (sanity check)
figure;
plot(t_win_centers, HR_win, 'b.-', 'LineWidth', 1.2, 'MarkerSize', 16); ho
plot(tHR, HR_inst, 'r.', 'MarkerSize', 12);
xlabel('Time (s)'); ylabel('HR (bpm)'); grid on;
legend('Target HR per window','Derived inst. HR','Location','best');
title('Target HR modulation vs. derived HR');
```

Why this fixes your error

- We **never** call `ecgsyn` with `N = 1`. Each call uses `N_beats >= 20`.